

OBJETIVOS

- I/O: teclado.
- Tratamento de interrupção
- Criação de um *shell* muito simples.

TAREFA 3: MODIFICAÇÃO DA ESTRUTURA DO PROJETO E BUILD (de 03/02/2025 a 06/02/2025)

Nesta tarefa, modificaremos a estrutura do projeto e seu mecanismo de construção (*build*) para utilizarmos o *make*. Na raiz do seu projeto, local onde se localiza o script *build.sh*, você irá criar o arquivo *Makefile*, o qual ficará responsável (em consonância com a ferramenta *make*) por gerenciar o processo de compilação e geração do *kernel* do SO.

make é uma ferramenta que permite a construção de um projeto de forma bastante “inteligente”. Inteligente aqui, refere-se ao fato de que, no caso do uso do *make*, se você compilar alguns arquivos e por ventura, atualizar um deles, apenas este será recompilado.

A organização do arquivo (Código 10) é bastante direta. No início do arquivo você tem a definição de diversas variáveis e seus valores (ex. *COMPILER*, *LINKER*, *ASSEMBLER* etc.). Para acessar o conteúdo da variável, basta usar a notação “*\$ (NOME DA VARIÁVEL)*”. Em sua organização básica, um arquivo *Makefile* possui um alvo (*target*) que você deseja construir e este, por sua vez possui uma dependência. Por exemplo, na entrada “*run: link*”, *run* é o alvo e *link* é a dependência. A dependência pode ser um alvo em outro ponto do arquivo. Faça as modificações apropriadas em sua árvore de diretórios e na declaração dos arquivos de cabeçalho (.h), realizadas nos arquivos-fonte .c de modo que o arquivo *Makefile* fornecido funcione para o seu trabalho. Você não deve alterar o arquivo *Makefile* fornecido. **Coloque informações suficientes em seu relatório de modo a identificar todas as modificações realizadas em seus códigos fonte/árvores de diretório para que o arquivo fornecido funcione.** Além disso, demonstre o seu entendimento sobre o significado do arquivo fornecido no Código 10.

```

COMPILER = gcc
LINKER = ld
ASSEMBLER = nasm
CFLAGS = -m32 -c -ffreestanding
ASFLAGS = -f elf32
LDFLAGS = -m elf_i386 -T src/link.ld
EMULATOR = qemu-system-i386
EMULATOR_FLAGS = -kernel

SRCS = src/kernel.asm src/kernel.c src/idt.c src/isr.c src/kb.c src/screen.c
src/string.c src/system.c src/util.c
OBJS = obj/kasm.o obj/kc.o obj/idt.o obj/isr.o obj/kb.o obj/screen.o
obj/string.o obj/system.o obj/util.o
OUTPUT = B.OS/boot/kernel.bin

run: link
    $(EMULATOR) $(EMULATOR_FLAGS) $(OUTPUT)

link: compile $(OBJS)
    rm -rf B.OS
    mkdir B.OS
    mkdir B.OS/boot
    $(LINKER) $(LDFLAGS) -o $(OUTPUT) $(OBJS)

compile: $(SRCS)
    rm obj/ -rf
    mkdir obj/

obj/kasm.o: src/kernel.asm
    $(ASSEMBLER) $(ASFLAGS) -o obj/kasm.o src/kernel.asm

obj/kc.o: src/kernel.c
    $(COMPILER) $(CFLAGS) src/kernel.c -o obj/kc.o

obj/idt.o: src/idt.c
    $(COMPILER) $(CFLAGS) src/idt.c -o obj/idt.o

obj/kb.o: src/kb.c
    $(COMPILER) $(CFLAGS) src/kb.c -o obj/kb.o

obj/isr.o: src/isr.c
    $(COMPILER) $(CFLAGS) src/isr.c -o obj/isr.o

obj/screen.o: src/screen.c
    $(COMPILER) $(CFLAGS) src/screen.c -o obj/screen.o

obj/string.o: src/string.c
    $(COMPILER) $(CFLAGS) src/string.c -o obj/string.o

obj/system.o: src/system.c
    $(COMPILER) $(CFLAGS) src/system.c -o obj/system.o

obj/util.o: src/util.c
    $(COMPILER) $(CFLAGS) src/util.c -o obj/util.o

build:
    # Activate the install xorrr if you do not have it already installed
    # sudo apt install xorrriso
    rm B.OS/boot/grub/ -rf
    mkdir B.OS/boot/grub
    echo set default=0 >> B.OS/boot/grub/grub.cfg
    echo set timeout=0 >> B.OS/boot/grub/grub.cfg
    echo menuentry "B.OS" { >> B.OS/boot/grub/grub.cfg

```

```
    echo  set root='(hd96)' >> B.OS/boot/grub/grub.cfg
    echo  multiboot /boot/kernel.bin >> B.OS/boot/grub/grub.cfg
    echo  } >> B.OS/boot/grub/grub.cfg
    grub-mkrescue -o B.OS.iso B.OS/

all: run build

clean:
    rm -rf obj/
    rm -rf B.OS/
    rm B.OS.iso
```

Código 10: Código para o Makefile.